

Module 8-9 Assignment:
Recursion, Iteration, and Algorithmic Performance Report

Alejandro Pantoja-Zurita

Delaware Technical Community College

CIS 211: Data Structures

Instructor: Jin An

April 26, 2026

Table of Contents

List of Figures	3
Hypothesis	4
Experimental Design	5
Results	6
Analysis	9
Conclusion	11

List of Figures

Graph 1: Factorial Runtime vs Input Size (Full Graph)	7
Graph 2: Factorial Runtime vs Input Size (20 Values)	8
Graph 3: Fibonacci Runtime vs Input Size Graph	9
Graph 4: Sorting Runtime Comparison Graph	10
Table 1: Factorial Runtime vs Input Size Table	7
Table 2: Fibonacci Runtime vs Input Size Table	8
Table 3: Sorting Runtime Comparison Table	9

Hypothesis

If asked to predict which would perform better between an algorithm that use recursion and one that uses iteration to solve a problem, I would initially assume that the iterative method would be more efficient. My reasoning for selecting iterative over recursive instinctively is because it is easier to grasp conceptually since it is progressive improvement starting at the beginning. In contrast, recursion approaches a problem from the beginning as well however it breaks it down into smaller task while working from the end result back to the beginning. Aside from that it also uses more overhead than the iterative process. That being said, after taking this class, if asked about determine runtime efficiency of the methods for determining the result of Fibonacci, the result of factorial and lastly determining which sorting algorithm is more efficient I would be able to provide a better answer. For factorial, I predict that the recursive method will be more efficient since the problem the complexity of the problem is reduced as you get closer to the value of 1; meaning that if we worked backwards it would go from simple to complex, which might result in improved efficiency. That being said, it is worth noting that the additional overhead when recursively calling the method may reduce performance, as of now I believe that this will not have a major impact on performance for obtaining the result of factorial. For Fibonacci, I predict that the iterative process may be faster for the opposite reason. It solves the problem linearly from the beginning until arriving at a solution; when it begins, the value of produced is small and grows larger as the value of n increases. Lastly for sorting, for comparison I looked at recursive merge sort and iterative selection sort. Out these two, I predict the recursive merge sort method would be faster since it is splitting up the problem into smaller solvable tasks. Furthermore, I know that selection sort requires sorting through the array of numbers completely

before the sorting can begin. This increases the time complexity of the method as the total number of values grow in size.

Experimental Design

When designing and setting up the test, I made sure to setup the test to meet the requirements listed for the assignment. For the Factorial and Fibonacci testing I was able to re-use most of the code with slight changes. The number of trials conducted per variable was the same, 5 times per value tested, to ensure there was accuracy for each value instead of just using the initial time provided for each value. The average time was stored and used the graphs provided below. For factorial, I required some tweaking to ensure input values provided accurate numbers. The first measurement appeared slower due to system initialization effects, it was excluded to improve consistency. With this in mind, I was initially going to use 20 different values per the instructions. (Graph 1). However, I revised it to include 21 instead. I removed the initial value that was the outlier so that I could gauge efficiency with more accuracy with 20 values. (Graph 2). The values were linear in fashion to try to avoid overflow if possible. Return values were extended to a safe range since the implementations provided for factorial would produce overflow due to return values being integers rather than long values. With regards to Fibonacci, the only thing that changed was the how values were selected, to prevent excessive runtime due to exponential growth, I increment the input size by 5. (Graph 3). With sorting time, I had to make sure that the sorting was comparable so I had to duplicate the array to keep testing consistent. I made sure to generate a new list of random integers through each test run per array sizes. Once again, there were 5 trials conducted per each array size to ensure fairness and accuracy. For all runtimes, `System.nanoTime()` was used to capture the starting time and

stopping. The runtime was computed based on the difference between these two. After 5 trials, for each test condition, the average was computed and recorded. (Graph 4).

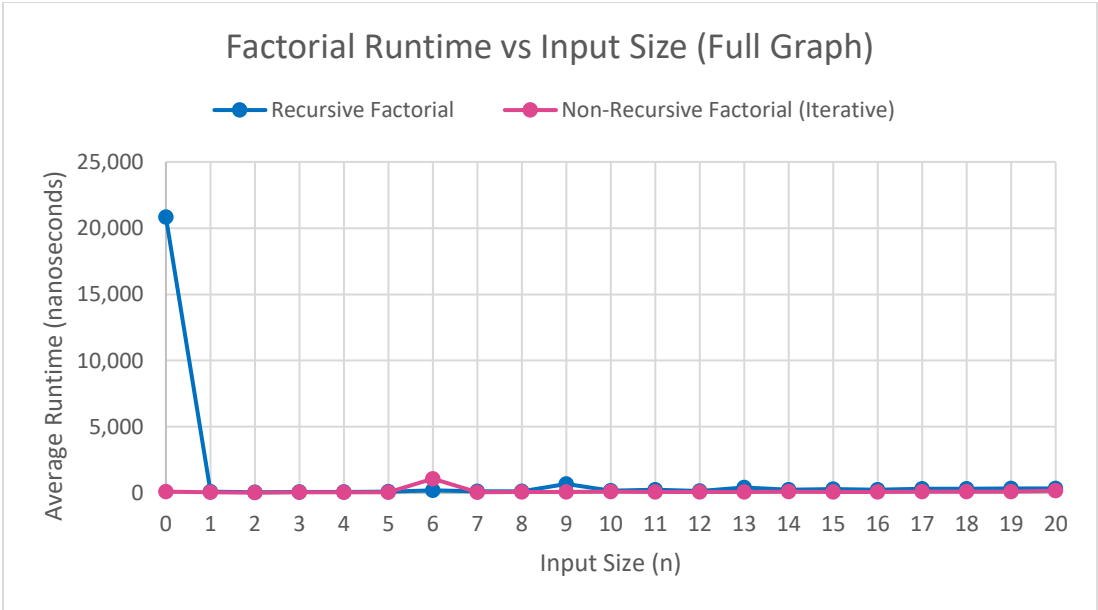
Results

Looking over the data for factorial testing, the average runtime was closer than anticipated. I believed that at some point the recursive method would be significantly faster than the iterative method. Looking at Table 1: Factorial Runtime vs Input Size Table we see that runtime was relatively stable across input sizes. There were a few outliers or bumps in consistency when looking at certain sections. Input size 0 seemed to produce a drastically higher runtime than any following values. (Graph 1: Factorial Runtime vs Input Size (Full Graph)). As mentioned before, the initial value was omitted to try to gauge more accurate values as demonstrated on Graph 2: Factorial Runtime vs Input Size (20 Values). There was a bump with the Iterative method at input size 6, meanwhile, with recursive method. There was a bump at input size 9 and 13. If these outliers were omitted, the completion time would reflect linear growth for both methods with Iterative proving to be slightly faster than recursive.

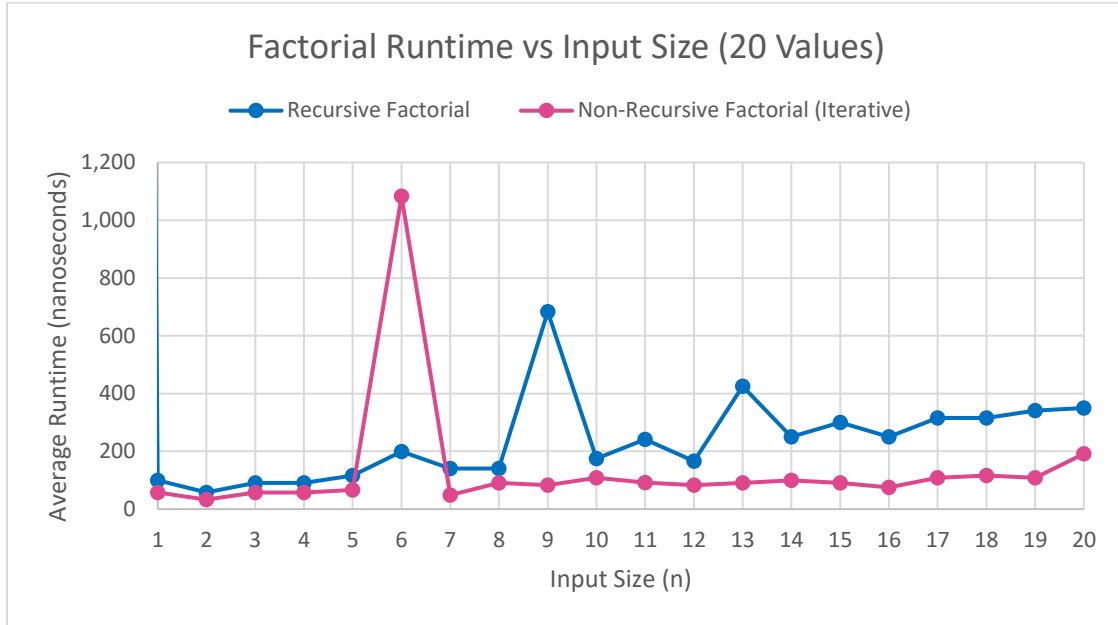
Table 1: Factorial Runtime vs Input Size Table

n	Recursive Factorial	Recursive Results	Non-Recursive Factorial (Iterative)	Iterative Results
0	20824	1	108	1
1	100	1	58	1
2	58	2	33	2
3	91	6	58	6
4	91	24	58	24
5	116	120	66	120
6	199	720	1083	720
7	141	5040	49	5040
8	141	40320	91	40320
9	683	362880	83	362880
10	175	3628800	108	3628800
11	241	39916800	92	39916800
12	166	479001600	83	479001600
13	425	6227020800	91	6227020800
14	250	87178291200	100	87178291200
15	300	1.30767E+12	91	1.30767E+12
16	250	2.09228E+13	75	2.09228E+13
17	316	3.55687E+14	108	3.55687E+14
18	316	6.40237E+15	116	6.40237E+15
19	341	1.21645E+17	108	1.21645E+17
20	350	2.4329E+18	191	2.4329E+18

Graph 1: Factorial Runtime vs Input Size (Full Graph)



Graph 2: Factorial Runtime vs Input Size (20 Values)

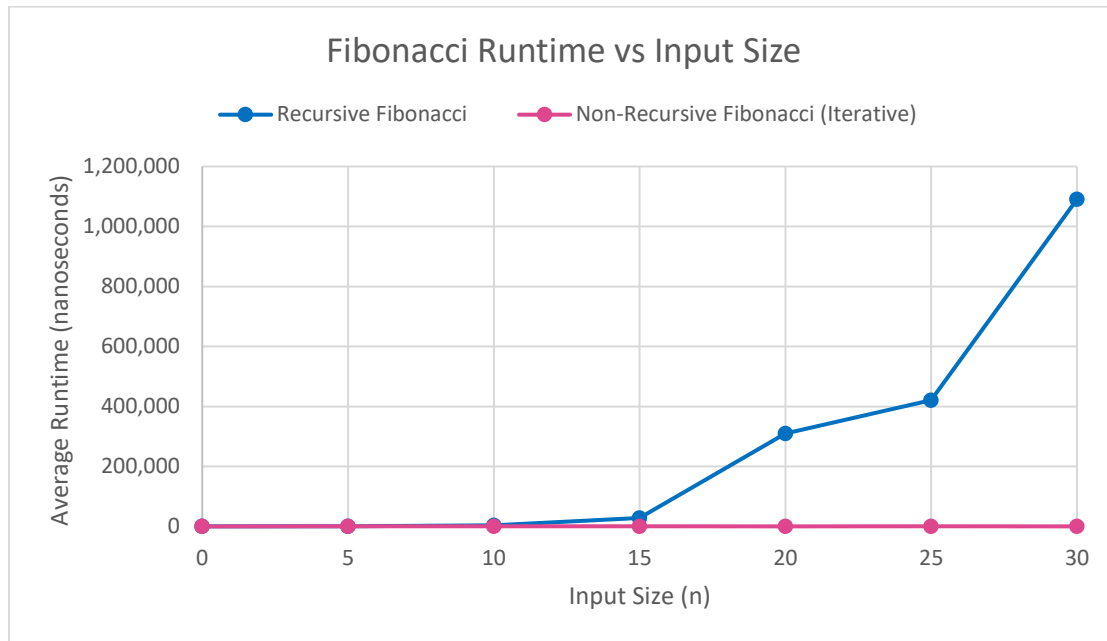


Looking over the Fibonacci testing data. The data shows that the recursive method's runtime grows faster than the iterative method as the input size increases. In comparison to the recursive method, the runtime for iterative method reflects linear growth, almost flat. When comparing both iterative and recursive methods, we can see that the Iterative method is generally faster than the recursive method regardless of value of n used.

Table 2: Fibonacci Runtime vs Input Size Table

n	Recursive Fibonacci	Recursive Results	Non-Recursive Fibonacci (Iterative)	Iterative Results
0	225	0	100	0
5	341	5	116	5
10	3500	55	133	55
15	28291	610	174	610
20	310083	6765	208	6765
25	421049	75025	183	75025
30	1090525	832040	208	832040

Graph 3: Fibonacci Runtime vs Input Size

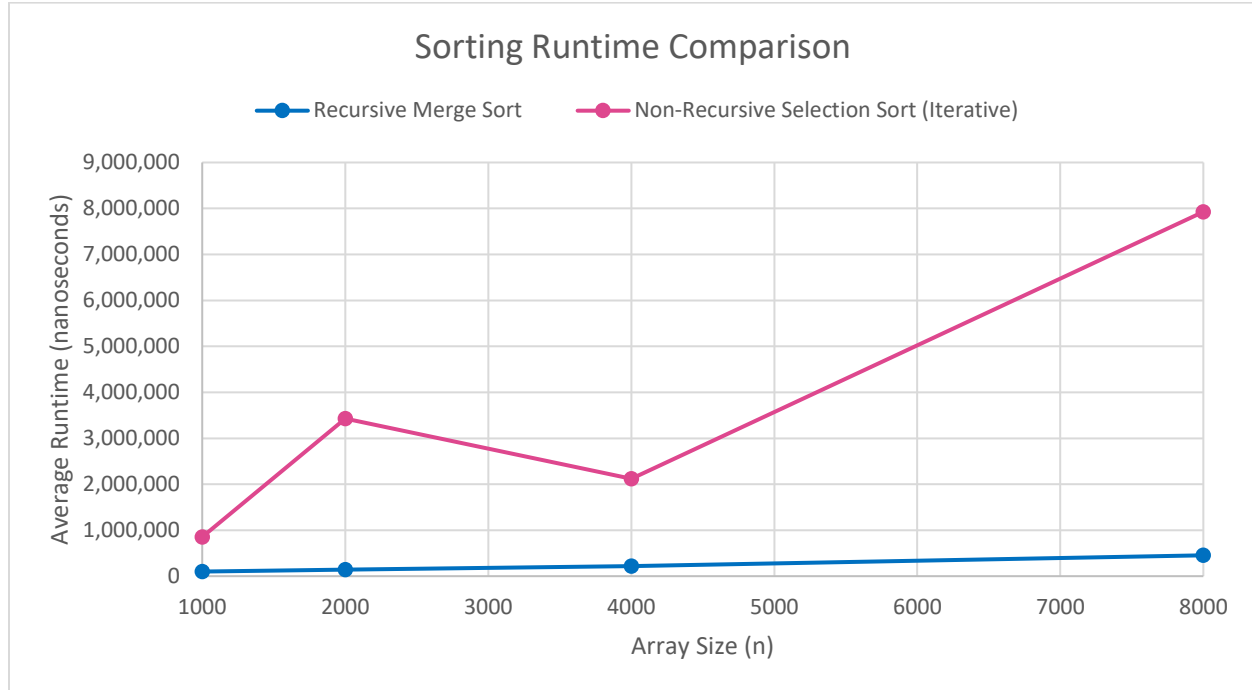


Lastly, Looking over the different sorting methods, recursive merge sort & iterative selection sort. The recursive merge sort method stays consistent in runtime; it also reflects more linear growth as the array size increases. In contrast, Iterative Selection Sort is be slower, the average runtime increases at a quadratic rate ($O(n^2)$) as the size of the array grows. This makes the slower method of sorting an array. When comparing the average runtimes for each method within the same array size, recursive merge sort is always faster than the iterative selection sort.

Table 3: Sorting Runtime Comparison Table

Array Size (n)	Recursive Merge Sort	Non-Recursive Selection Sort (Iterative)
1000	101299	927433
2000	137508	3240141
4000	224308	2314208
8000	443625	8299933

Graph 4: Sorting Runtime Comparison Graph



Analysis

Looking over the data produced, the results were consistent with initial expectations. The outlier to my predictions was with the factorial testing. I predicted that the recursive method would be faster than the iterative method. This proved to be incorrect when analyzing the data; that being said, they were closer than I had anticipated. While the iterative method faster with factorials, both methods can be used interchangeably since performance is relatively the same. Reflecting over the data, I believe the slight difference in performance is a result of the overhead when solving the problem recursively. Following that, Fibonacci testing is next; this falls within predicted outcomes. The iterative method is consistent with runtime, whereas the recursive method's runtime grows exponentially. This is a result of the work that is performed within each method. The recursive method breaks the problem into smaller subproblems, repeatedly recomputing the same values, resulting in exponential time complexity ($O(2^n)$). This leads to repeated work across recursive calls, significantly increasing runtime. The iterative method

approaches the problem differently, working on the problem as it goes, eliminating the need for work to be duplicated, thus improving performance time from $O(2^n)$ to $O(n)$. Lastly, looking the runtime for the sorting methods, results fall within predicted outcomes. With recursive merge sort being the clear winner. This is a result of the approach to sorting. Merge sort splits the problem in two to reduce the amount variables that must be evaluated at once. This improves performance in organizing a set of numbers before combining both into a final set of organized values $O(n \log(n))$. Meanwhile, selection sort requires the list to be iterated through entirely at least once in order to find the smallest value possible to place that value the sorted section. Once this is done, the process is repeated again with second smallest value and so on, each time traversing the list to find said value. This approach results in a longer sorting times since the number of comparisons grows significantly as the input size increases ($O(n^2)$). Merge sort uses the divide and conquer method to solve the problem faster than trying to brute force the sorting. The AlgorithmLibrary also provided binary search methods, both iterative and recursive. Binary search was not included in timing experiments because they provide the same time complexity regardless of implementation, $O(\log(n))$. Regardless of implementation, binary search has a divide and conquer approach to searching for a value.

Conclusion

Apart from the factorial, my predictions stated in my hypotheses section were correct. The testing conducted helped visualize why recursion can be beneficial, although not always, depending on the problem can drastically reduce performance. When choosing between recursion and an iterative approach to problem solving, it is important to consider if the problem can be reduced in complexity as a result of splitting the work up into smaller, more-workable problems. Fibonacci demonstrated that recursion without optimization can lead to poor

performance due to repeated work. Meanwhile, merge sort demonstrated the benefits of recursion. Factorial provides another key to understanding when recursion may prove beneficial. For the factorial testing, both the recursive and iterative approaches achieved similar runtimes. However, the recursive approach was slightly slower, this could be a result of the required overhead for the recursive method. Thus, when looking at a problem you must also consider the overhead available to solve the problem as it can impact performance as well.